Teach Module 29: Validation and Global Exception Handling

This note was created from our Module 29 teaching chat. The course document was used only to identify the module topic. The teaching content below is independent explanatory material.

Module Focus

Module 29 covers validation and global API error handling in Spring Boot:

- Request validation with Bean Validation
- Global exception handling with `@ControllerAdvice` or `@RestControllerAdvice`
- Standard API error responses
- Different response shapes for different error types
- Logging strategy for errors
- Lab: validation and exception handling

Big Idea

In a Spring Boot API, users send data to your backend. That data may be wrong, incomplete, unsafe, or inconsistent.

So we need two things:

1. Validation: reject bad input before business logic runs.
2. Exception handling: return clean, predictable error responses when something goes wrong.

Without this, APIs often return messy errors like stack traces, vague `500` responses, or inconsistent messages.

Good APIs behave like this:

```
{
  "timestamp": "2026-08-02T10:15:30",
  "status": 400,
  "error": "Validation Failed",
  "message": "Request contains invalid fields",
  "details": {
    "email": "must be a valid email address",
    "name": "must not be blank"
  }
}
```

That is much better than exposing raw Java exception details.

Bean Validation

Spring Boot commonly uses Jakarta Bean Validation.

Example request DTO:

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
```

```java
import jakarta.validation.constraints.Size;

public class CreateUserRequest {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be between 2 and 50 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;

    // getters and setters
}
```

Common annotations:

```java
@NotNull      // field must not be null
@NotBlank     // string must not be null, empty, or whitespace
@NotEmpty     // collection/string must not be empty
@Size         // length or collection size
@Min          // minimum numeric value
@Max          // maximum numeric value
@Email        // valid email format
@Pattern      // regex validation
```

Then in the controller:

```java
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {

    @PostMapping
    public UserResponse createUser(@Valid @RequestBody CreateUserRequest request) {
        return userService.createUser(request);
    }
}
```

The important part is:

```java
@Valid @RequestBody CreateUserRequest request
```

`@Valid` tells Spring: before entering this method, validate the request object.

If validation fails, Spring throws an exception automatically.

Why Validation Belongs In DTOs

Avoid putting request validation directly in the controller like this:

```
if (request.getEmail() == null) {
    throw new RuntimeException("Email required");
}
```

That gets messy quickly.

Better:

```
public class CreateUserRequest {
    @NotBlank
    @Email
    private String email;
}
```

This keeps validation close to the data contract.

```
Controller = receives request
DTO = defines input rules
Service = business logic
Exception handler = error response formatting
```

Global Exception Handling

Instead of writing `try/catch` in every controller, Spring lets us centralize error handling using:

```
@ControllerAdvice
```

or commonly:

```
@RestControllerAdvice
```

Example:

```
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleUserNotFound(UserNotFoundException ex) {
        return new ErrorResponse(
            404,
```

```
            "User Not Found",
            ex.getMessage()
        );
    }
}
```

Now any controller that throws `UserNotFoundException` will automatically return a clean `404`.

Standard Error Response

Create a reusable error model:

```
import java.time.LocalDateTime;

public class ErrorResponse {

    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;

    public ErrorResponse(int status, String error, String message) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = error;
        this.message = message;
    }

    // getters
}
```

Example output:

```
{
  "timestamp": "2026-08-02T14:22:10",
  "status": 404,
  "error": "User Not Found",
  "message": "User with id 25 was not found"
}
```

Handling Validation Errors

When `@Valid` fails, Spring usually throws `MethodArgumentNotValidException`.

Handle it like this:

```
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
```

```java
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ValidationErrorResponse handleValidationErrors(MethodArgumentNotValidException
ex) {

        Map<String, String> fieldErrors = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error -> {
            fieldErrors.put(error.getField(), error.getDefaultMessage());
        });

        return new ValidationErrorResponse(
            LocalDateTime.now(),
            400,
            "Validation Failed",
            "Request contains invalid fields",
            fieldErrors
        );
    }
}
```

Example response:

```json
{
  "timestamp": "2026-08-02T14:30:00",
  "status": 400,
  "error": "Validation Failed",
  "message": "Request contains invalid fields",
  "fieldErrors": {
    "name": "Name is required",
    "email": "Email must be valid"
  }
}
```

This is very useful for frontend teams because they can show field-specific messages.

Different Error Types

A mature API usually distinguishes these:

```
400 Bad Request       - invalid input
401 Unauthorized      - user is not authenticated
403 Forbidden         - user is authenticated but not allowed
404 Not Found         - resource does not exist
```

```
409 Conflict          - business conflict, duplicate data, invalid state
500 Internal Error    - unexpected server failure
```

Example custom exception:

```
public class DuplicateEmailException extends RuntimeException {

    public DuplicateEmailException(String email) {
        super("Email already exists: " + email);
    }
}
```

Handler:

```
@ExceptionHandler(DuplicateEmailException.class)
@ResponseStatus(HttpStatus.CONFLICT)
public ErrorResponse handleDuplicateEmail(DuplicateEmailException ex) {
    return new ErrorResponse(
        409,
        "Duplicate Email",
        ex.getMessage()
    );
}
```

Logging Strategy

Do not log every error the same way.

Use levels intentionally:

```
INFO   - normal business event
WARN   - expected problem, such as invalid user input or missing resource
ERROR  - unexpected failure requiring investigation
```

Example:

```
private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleUnexpectedError(Exception ex) {
    log.error("Unexpected server error", ex);

    return new ErrorResponse(
        500,
        "Internal Server Error",
        "Something went wrong. Please try again later."
    );
}
```

Important rule:

Do not expose internal exception details to API users.

Bad:

```
{
  "message": "NullPointerException at UserService.java:42"
}
```

Better:

```
{
  "message": "Something went wrong. Please try again later."
}
```

The full stack trace belongs in logs, not in the API response.

Typical Project Structure

A clean structure might look like this:

```
src/main/java/com/example/app
  controller/
    UserController.java
  dto/
    CreateUserRequest.java
    UserResponse.java
  exception/
    GlobalExceptionHandler.java
    ErrorResponse.java
    ValidationErrorResponse.java
    UserNotFoundException.java
    DuplicateEmailException.java
  service/
    UserService.java
```

This keeps exception handling reusable across the whole application.

Practice Exercises

Exercise 1: Validate A Product Request

Create a `CreateProductRequest` DTO with:

```
@NotBlank
private String name;

@NotNull
@DecimalMin(value = "0.01")
```

```
private BigDecimal price;

@NotBlank
@Size(min = 4, max = 20)
private String sku;

@Size(max = 500)
private String description;
```

Then create a `POST /products` endpoint using:

```
@Valid @RequestBody CreateProductRequest request
```

Test bad requests such as missing name, negative price, short SKU, and long description.

Exercise 2: Return Field-Level Validation Errors

Build a global exception handler for `MethodArgumentNotValidException`.

Goal response:

```
{
  "status": 400,
  "error": "Validation Failed",
  "message": "Request contains invalid fields",
  "fieldErrors": {
    "name": "must not be blank",
    "price": "must be greater than or equal to 0.01"
  }
}
```

Exercise 3: Create Custom Exceptions

Create custom exceptions:

```
ProductNotFoundException
DuplicateSkuException
InvalidProductStateException
```

Map them to:

```
ProductNotFoundException       -> 404 Not Found
DuplicateSkuException          -> 409 Conflict
InvalidProductStateException   -> 400 Bad Request
```

Then throw them from a service class and confirm your API returns the correct HTTP status.

Exercise 4: Standardize Error Responses

Create one reusable error class:

```
public class ApiErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
}
```

Use it for all non-validation exceptions.

Bonus: include the request path using `HttpServletRequest`.

Exercise 5: Add A Catch-All Handler

Add a final fallback handler:

```
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ApiErrorResponse handleUnexpected(Exception ex) {
    // log exception
    // return safe message
}
```

The API response should not expose stack traces or Java class names.

Exercise 6: Logging Practice

Use different log levels:

```
WARN  - validation failures, resource not found, duplicate data
ERROR - unexpected server errors
```

Practice logging the real exception internally while returning a user-safe message externally.

Exercise 7: Test With Postman Or curl

Send bad requests like:

```
{
  "name": "",
  "price": -10,
  "sku": "A",
  "description": "very long text..."
}
```

Confirm:

```
HTTP status is 400
Response is JSON
```

```
Each invalid field has a clear message
No stack trace is exposed
```

Exercise 8: Write Controller Tests

Use `MockMvc` to test validation behavior.

Example test goals:

```
POST /products with blank name returns 400
POST /products with invalid price returns 400
POST /products with duplicate SKU returns 409
GET /products/{id} for missing product returns 404
Unexpected exception returns 500
```

Lab: Validation And Global Exception Handling

Build a small Spring Boot Product API that validates incoming requests and returns standardized error responses.

Lab Goal

Create an API where invalid requests return clean `400` responses, missing products return `404`, duplicate SKUs return `409`, and unexpected errors return safe `500` responses.

Part 1: Create The DTO

Create `CreateProductRequest`:

```java
public class CreateProductRequest {

    @NotBlank(message = "Product name is required")
    @Size(min = 2, max = 100, message = "Product name must be between 2 and 100 characters")
    private String name;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.01", message = "Price must be greater than 0")
    private BigDecimal price;

    @NotBlank(message = "SKU is required")
    @Size(min = 4, max = 20, message = "SKU must be between 4 and 20 characters")
    private String sku;

    @Size(max = 500, message = "Description cannot exceed 500 characters")
    private String description;
}
```

Use getters/setters or Lombok.

Part 2: Create The Controller

Create:

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @PostMapping
    public String createProduct(@Valid @RequestBody CreateProductRequest request) {
        return "Product created successfully";
    }
}
```

Test with invalid JSON and confirm Spring rejects the request before the method completes.

Part 3: Create Error Response Classes

Create a standard error response:

```
public class ApiErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
}
```

Create a validation error response:

```
public class ValidationErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private Map<String, String> fieldErrors;
}
```

Part 4: Create Custom Exceptions

```
public class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(Long id) {
        super("Product with id " + id + " was not found");
    }
}
```

```
public class DuplicateSkuException extends RuntimeException {
    public DuplicateSkuException(String sku) {
        super("Product with SKU " + sku + " already exists");
    }
}
```

Part 5: Create Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ValidationErrorResponse handleValidationErrors(
            MethodArgumentNotValidException ex,
            HttpServletRequest request
    ) {
        Map<String, String> fieldErrors = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error ->
                fieldErrors.put(error.getField(), error.getDefaultMessage())
        );

        return new ValidationErrorResponse(
                LocalDateTime.now(),
                400,
                "Validation Failed",
                "Request contains invalid fields",
                request.getRequestURI(),
                fieldErrors
        );
    }

    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiErrorResponse handleProductNotFound(
            ProductNotFoundException ex,
            HttpServletRequest request
    ) {
        return new ApiErrorResponse(
                LocalDateTime.now(),
                404,
                "Product Not Found",
                ex.getMessage(),
                request.getRequestURI()
        );
    }

    @ExceptionHandler(DuplicateSkuException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ApiErrorResponse handleDuplicateSku(
            DuplicateSkuException ex,
            HttpServletRequest request
    ) {
        return new ApiErrorResponse(
                LocalDateTime.now(),
                409,
```

```
                "Duplicate SKU",
                ex.getMessage(),
                request.getRequestURI()
        );
    }
}
```

Part 6: Add Test Endpoints

Temporarily add these methods to test exception handling:

```
@GetMapping("/{id}")
public String getProduct(@PathVariable Long id) {
    throw new ProductNotFoundException(id);
}
```

```
@PostMapping("/duplicate")
public String duplicateSku() {
    throw new DuplicateSkuException("ABC123");
}
```

```
@GetMapping("/error")
public String serverError() {
    throw new RuntimeException("Database connection failed");
}
```

Then add a catch-all handler:

```
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ApiErrorResponse handleUnexpectedError(
        Exception ex,
        HttpServletRequest request
) {
    return new ApiErrorResponse(
            LocalDateTime.now(),
            500,
            "Internal Server Error",
            "Something went wrong. Please try again later.",
            request.getRequestURI()
    );
}
```

Expected Tests

Send this invalid request:

```
POST /products
Content-Type: application/json
```

```
{
  "name": "",
  "price": -5,
  "sku": "A",
  "description": ""
}
```

Expected response: `400 Bad Request`

```
{
  "status": 400,
  "error": "Validation Failed",
  "message": "Request contains invalid fields",
  "path": "/products",
  "fieldErrors": {
    "name": "Product name is required",
    "price": "Price must be greater than 0",
    "sku": "SKU must be between 4 and 20 characters"
  }
}
```

Also test:

```
GET /products/99         -> 404
POST /products/duplicate -> 409
GET /products/error      -> 500
```

Completion Criteria

You are done when:

```
Invalid request bodies return 400
Validation errors identify individual fields
Missing product errors return 404
Duplicate SKU errors return 409
Unexpected errors return 500
No stack trace appears in the API response
All error responses follow the same structure
```

Best Practice Challenge

Build a small API with these endpoints:

```
POST /products
GET /products/{id}
PUT /products/{id}
DELETE /products/{id}
```

Add validation and global exception handling so every error response is predictable and consistent.

That one exercise gives you the full Module 29 muscle memory.